

Blocking state of Task

// main.c

```
/**
*****
* @file      main.c
* @author    Sagar More
* @brief     Minimal RTOS Scheduler using SysTick + PendSV + task_delay
*****
*/

#include "main.h"
#include "led.h"

/* ===== TASK PROTOTYPES ===== */
void task1_handler(void);
void task2_handler(void);
void task3_handler(void);
void task4_handler(void);
void ideal_task(void);

/* ===== SCHEDULER PROTOTYPES ===== */
void init_tasks_stack(void);
void init_systick(uint32_t tick_hz);
void switch_sp_to_psp(void);

uint32_t get_psp_value(void);
void save_psp_value(uint32_t psp);
void update_next_task(void);
void task_delay(uint32_t tick_count);
void unblock_tasks(void);
void update_global_tick_count(void);
void schedule(void);

/* ===== GLOBAL VARIABLES ===== */

uint8_t  current_task = 1;    // 0 = idle
uint32_t g_tick_count = 0;

/* ===== TCB ===== */

typedef struct {
    uint32_t psp_value;
    uint32_t block_count;
    uint8_t  current_state;
    void (*task_handler)(void);
} TCB_t;

TCB_t user_tasks[MAX_TASKS];
```

```

/* ===== MAIN ===== */

int main(void)
{
    /* Set MSP for scheduler */
    __asm volatile ("MSR MSP, %0" :: "r"(SCHED_STACK_START));

    /* PendSV lowest priority */
    NVIC_SHPR3 |= (0xFF << 16);

    init_tasks_stack();
    led_init_all();
    init_systick(TICK_HZ);

    switch_sp_to_psp();
    schedule(); // start scheduler

    while (1);
}

/* ===== SYSTICK ===== */

void update_global_tick_count(void)
{
    g_tick_count++;
}

void unblock_tasks(void)
{
    for (int i = 1; i < MAX_TASKS; i++)
    {
        if (user_tasks[i].current_state == TASK_BLOCKED_STATE &&
            user_tasks[i].block_count <= g_tick_count)
        {
            user_tasks[i].current_state = TASK_READY_STATE;
        }
    }
}

void SysTick_Handler(void)
{
    update_global_tick_count();
    unblock_tasks();
    schedule();
}

/* ===== PENDSV ===== */

__attribute__((naked)) void PendSV_Handler(void)
{
    __asm volatile (
        "MRS R0, PSP          \n"
        "STMDB R0!, {R4-R11}  \n"
        "BL save_psp_value    \n"
    );

    update_next_task();
}

```

```

    __asm volatile (
        "BL get_psp_value      \n"
        "LDMIA R0!, {R4-R11}   \n"
        "MSR PSP, R0           \n"
        "BX LR                  \n"
    );
}

/* ===== TASK SELECTION ===== */

void update_next_task(void)
{
    for (int i = 0; i < MAX_TASKS; i++)
    {
        current_task++;
        current_task %= MAX_TASKS;

        if (user_tasks[current_task].current_state == TASK_READY_STATE)
            return;
    }

    current_task = 0; // idle fallback
}

/* ===== TASK STACK INIT ===== */

void init_tasks_stack(void)
{
    uint32_t *psp;

    user_tasks[0] = (TCB_t){IDEAL_STACK_START, 0, TASK_READY_STATE,
ideal_task};
    user_tasks[1] = (TCB_t){T1_STACK_START, 0, TASK_READY_STATE,
task1_handler};
    user_tasks[2] = (TCB_t){T2_STACK_START, 0, TASK_READY_STATE,
task2_handler};
    user_tasks[3] = (TCB_t){T3_STACK_START, 0, TASK_READY_STATE,
task3_handler};
    user_tasks[4] = (TCB_t){T4_STACK_START, 0, TASK_READY_STATE,
task4_handler};

    for (int i = 0; i < MAX_TASKS; i++)
    {
        psp = (uint32_t *)user_tasks[i].psp_value;

        *(--psp) = DUMMY_XPSR;
        *(--psp) = (uint32_t)user_tasks[i].task_handler;
        *(--psp) = 0xFFFFFFF;
        *(--psp) = 0; *(--psp) = 0; *(--psp) = 0;
        *(--psp) = 0; *(--psp) = 0;

        for (int j = 0; j < 8; j++)
            *(--psp) = 0;

        user_tasks[i].psp_value = (uint32_t)psp;
    }
}

```

```

/* ===== PSP SWITCH ===== */

__attribute__((naked)) void switch_sp_to_psp(void)
{
    __asm volatile (
        "BL get_psp_value  \n"
        "MSR PSP, R0       \n"
        "MOV R0, #0x02     \n"
        "MSR CONTROL, R0   \n"
        "BX LR              \n"
    );
}

/* ===== SYSTICK INIT ===== */

void init_systick(uint32_t tick_hz)
{
    uint32_t count = (SYSTICK_TIM_CLK / tick_hz) - 1;
    *(uint32_t *)0xE000E014 = count;
    *(uint32_t *)0xE000E010 = 7;
}

/* ===== TASKS ===== */

void ideal_task(void)
{
    while (1)
    {
        __asm volatile ("WFI");
    }
}

void task1_handler(void)
{
    while (1) {
        led_on(LED_GREEN);
        task_delay(1000);
        led_off(LED_GREEN);
        task_delay(1000);
    }
}

void task2_handler(void)
{
    while (1) {
        led_on(LED_ORANGE);
        task_delay(500);
        led_off(LED_ORANGE);
        task_delay(500);
    }
}

void task3_handler(void)
{
    while (1) {
        led_on(LED_BLUE);
        task_delay(250);
    }
}

```

```

        led_off(LED_BLUE);
        task_delay(250);
    }
}

void task4_handler(void)
{
    while (1) {
        led_on(LED_RED);
        task_delay(125);
        led_off(LED_RED);
        task_delay(125);
    }
}

/* ===== DELAY ===== */

void task_delay(uint32_t tick_count)
{
    INTERRUPT_DISABLE();

    if (current_task != 0)
    {
        user_tasks[current_task].block_count = g_tick_count +
tick_count;
        user_tasks[current_task].current_state = TASK_BLOCKED_STATE;
        schedule();
    }

    INTERRUPT_ENABLE();
}

/* ===== SCHEDULER ===== */

void schedule(void)
{
    ICSR |= (1 << 28);
}

/* ===== PSP HELPERS ===== */

uint32_t get_psp_value(void)
{
    return user_tasks[current_task].psp_value;
}

void save_psp_value(uint32_t psp)
{
    user_tasks[current_task].psp_value = psp;
}

```

// led.c

```
/*
 * led.c
 *
 * Created on: Jan 30, 2026
 * Author: Lenovo
 */

#include "led.h"
#include <stdint.h>

void led_init_all(void){
    RCC_AHB1_ENR |= (1 << 3);

    GPIO_MODE_REG |= (1 << (2 * LED_GREEN));
    GPIO_MODE_REG |= (1 << (2 * LED_ORANGE));
    GPIO_MODE_REG |= (1 << (2 * LED_RED));
    GPIO_MODE_REG |= (1 << (2 * LED_BLUE));

    // Make all LED to off
    led_off(LED_GREEN);
    led_off(LED_BLUE);
    led_off(LED_ORANGE);
    led_off(LED_RED);
}

void led_on(uint8_t led_no){
    GPIO_DATA_REG |= (1 << led_no);
}

void led_off(uint8_t led_no){
    GPIO_DATA_REG |= ~(1 << led_no);
}
```

// main.h

```
#ifndef MAIN_H_
#define MAIN_H_

#include <stdint.h>

/* ===== STACK SIZES ===== */

#define SIZE_TASK_STACK      1024U
#define SIZE_SCHEDULER_STACK 1024U

/* ===== SRAM ===== */
```

```

#define SRAM_START                0x20000000U
#define SRAM_SIZE                 (128U * 1024U)
#define SRAM_END                 (SRAM_START + SRAM_SIZE)

/*
 * Stack layout (TOP → DOWN)
 */
/* |-----| 0x20020000 <- MSP (Scheduler)
 * | Scheduler Stack |
 * |-----|
 * | Idle Task Stack |
 * |-----|
 * | Task1 Stack |
 * |-----|
 * | Task2 Stack |
 * |-----|
 * | Task3 Stack |
 * |-----|
 * | Task4 Stack |
 * |-----|
 */

#define SCHED_STACK_START        SRAM_END
#define IDEAL_STACK_START        (SRAM_END - 1U * SIZE_TASK_STACK)
#define T1_STACK_START           (SRAM_END - 2U * SIZE_TASK_STACK)
#define T2_STACK_START           (SRAM_END - 3U * SIZE_TASK_STACK)
#define T3_STACK_START           (SRAM_END - 4U * SIZE_TASK_STACK)
#define T4_STACK_START           (SRAM_END - 5U * SIZE_TASK_STACK)

/* ===== SYSTICK ===== */

#define TICK_HZ                   1000U
#define HSI_CLK                   16000000U
#define SYSTICK_TIM_CLK           HSI_CLK

/* ===== SCHEDULER ===== */

#define MAX_TASKS                 5U

#define TASK_READY_STATE          0x00
#define TASK_BLOCKED_STATE        0xFF

/* ===== CPU ===== */

#define DUMMY_XPSR                0x01000000U /* Thumb bit */

/* ===== REGISTERS ===== */

#define ICSR                      (*(volatile uint32_t*)0xE00ED04)
#define SHCSR                     (*(volatile uint32_t*)0xE00ED24)
#define NVIC_SHPR3                (*(volatile uint32_t*)0xE00ED20)

/* ===== INTERRUPT CONTROL ===== */

#define INTERRUPT_DISABLE() \
    do { __asm volatile ("CPSID I"); } while (0)

```

```

#define INTERRUPT_ENABLE() \
    do { __asm volatile ("CPSIE I"); } while (0)

#endif /* MAIN_H_ */

```

// led.h

```

/*
 * led.h
 *
 * Created on: Jan 30, 2026
 * Author: Lenovo
 */

#ifndef LED_H_
#define LED_H_
#include <stdint.h>

#define RCC_AHB1_ENR (*(uint32_t*) 0x40023830)
#define GPIO_MODE_REG (*(uint32_t*) 0x40020C00)
#define GPIO_DATA_REG (*(uint32_t*) 0x40020C14)

#define LED_GREEN 12
#define LED_ORANGE 13
#define LED_RED 14
#define LED_BLUE 15

#define DELAY_COUNT_1MS 1250U
#define DELAY_COUNT_1S (1000U * DELAY_COUNT_1MS)
#define DELAY_COUNT_500MS (500U * DELAY_COUNT_1MS)
#define DELAY_COUNT_250MS (250U * DELAY_COUNT_1MS)
#define DELAY_COUNT_125MS (125U * DELAY_COUNT_1MS)

void led_init_all(void);
void led_on(uint8_t led_no);
void led_off(uint8_t led_no);
void delay(uint32_t count);

#endif /* LED_H_ */

```


